Anti-Pattern: Detect-Resolve-Forget — Standalone Formalization of the Failure Mode Where Conflicts Are Detected and Resolved But Then Removed From Substrate, Violating Substrate-Level Preservation Retroactively in CKS

Author: Wenxin Li (Independent Researcher)
ORCID: 0009-0004-8065-3235
Date: May 6, 2026

Attribution

This work derives from and formalizes the Coordination Knowledge Substrate (CKS) pattern introduced in *Coordination Outside the Model: A Human-Governed Substrate Pattern for AI Systems* (Li, April 2026). It does not introduce new axioms. Its sole contribution is to formalize, in operational form, a specific architectural anti-pattern — *detect-resolve-forget* — as a standalone object of public prior art, distinct from but adjacent to A3.08 (silent conflict resolution) and A3.10 (contradiction collapse by automation).

Abstract

The CKS pattern commits to conflicts as first-class substrate objects under a two-level handling discipline (substrate-level preservation, cell-level resolution under orchestration rules). *Detect-resolve-forget* is the architectural failure mode in which a deployment honors the first-level commitment transiently — producing a contradiction edge at conflict emergence and recording conflict provenance — and then undoes that preservation after resolution by removing the contradiction edge from substrate. The substrate state that remains contains the resolution outcome but not the conflict that produced it; humans exercising the inspect right cannot reconstruct what was in conflict before resolution; the retraceable trail breaks at the forget moment. This note formalizes the anti-pattern as four operational components, identifies the CKS commitments it violates (A1.03, A2.13, A2.17, A2.45, A1.07 directly; A2.16 partially; A2.36–A2.39, A1.10, A1.01 by cascade), traces the failure mode, specifies the architectural correction (persistent preservation, status-marking in place of removal, archival as alternative to deletion), distinguishes the anti-pattern from four adjacent legitimate patterns, and provides an operational test with three sharpening properties.

1. Why this anti-pattern needs standalone formalization

The CKS commitment to conflicts as first-class substrate objects (A1.03) is architectural and persistent: the contradiction edge produced at conflict emergence (A2.17) and the conflict provenance recorded with it (A2.16) are commitments to substrate state that holds throughout the deployment's existence, not only at the moment of conflict detection. *Detect-resolve-forget* is the failure mode where this persistence is undone after resolution. Motivating cases are operationally common: a conflict-resolution

interface that lets a user pick a resolution and removes the conflict from active substrate; a workflow that flags conflicts for review, resolves them under rules, and deletes the conflict record; a periodic cleanup that prunes "resolved" contradiction edges; garbage collection that removes conflicts past a time threshold.

Conflict-resolution UIs and data-cleanup processes are operationally attractive as positive practices ("we resolve conflicts and clean them up"), and the architectural cost — loss of historical retraceability, loss of conflict-context for resolution outcomes, loss of substrate authority over historical conflict state — is not visible without the anti-pattern named. Standalone formalization draws the line between cleanup that preserves conflict information (legitimate) and cleanup that erases it (the anti-pattern), and disambiguates this failure mode from sibling A3.08 (silent conflict resolution, which fails preservation from the start) and A3.10 (contradiction collapse by automation, the systematic-automation variant of forgetting at scale).

2. The anti-pattern, defined

A deployment exhibits *detect-resolve-forget* when its handling of substrate-level conflict state has the following four operational components, jointly or in any combination.

Component (a) — Transient contradiction edges that are removed after resolution. Contradiction edges per A2.17 are produced at conflict emergence and removed from substrate after resolution. The edges have a transient lifecycle — created at detection, deleted after resolution — rather than persisting as substrate state throughout the deployment's existence.

Component (b) — Conflict-cleanup processes that prune resolved conflicts. The deployment includes operational processes that actively remove resolved conflicts from substrate: UI-driven (a "clear" or "dismiss" action that the human takes after resolving), workflow-driven (cleanup as a step in the resolution workflow), or schedule-driven (periodic operations).

Component (c) — Garbage-collection of "old" conflicts past time thresholds. The deployment includes garbage-collection logic that removes conflicts older than some time threshold, treating resolved conflicts as transient operational state rather than as historical substrate content.

Component (d) — Resolution outcomes that replace conflict edges rather than coexist with them. The deployment records resolution outcomes (the chosen value, the winning source, the agreed-upon content) by replacing the conflict edge rather than by persisting alongside it. The substrate after resolution contains the outcome but not the conflict that produced it.

The four components are operationally severable. A deployment exhibiting any one component partially exhibits the anti-pattern; a deployment exhibiting all four exhibits it fully.

3. Which CKS commitments are violated

Direct violations are temporally precise — they hold from the forget moment forward — and cascade-implications follow.

A1.03 (conflict as first-class object) — directly violated retroactively. The first-class status holds transiently — at conflict emergence and through resolution — but is undone after resolution. The commitment requires persistence, not transient first-class status.

A2.13 (substrate-level conflict preservation) — directly violated retroactively. A2.13 specifies preservation throughout deployment existence; detect-resolve-forget preserves transiently and then undoes preservation at edge removal.

A2.17 (signed contradiction edge) — directly violated by removal. A2.17 commits contradiction edges as architectural objects that persist as substrate state. The persistence is the failure point: edges may be produced and may carry provenance at emergence, but they do not remain substrate-resident.

A2.45 (source-of-truth for conflict information) — directly violated historically. A2.45 commits substrate to being authoritative for what is in conflict. Detect-resolve-forget produces substrate states where past conflicts are not recorded; substrate is therefore not authoritative for historical conflict information.

A1.07 (path retraceability) — directly violated at the forget moment. The retraceable trail per A1.07 records substrate state changes including conflict emergence, conflict resolution, and the relationship between them. Detect-resolve-forget breaks the trail at the forget moment because the edges and provenance fields that anchor it to the conflict's existence are removed; subsequent retracing cannot reconstruct the conflict-and-resolution segment.

A2.16 (conflict provenance metadata) — partially compromised. Detect-resolve-forget records provenance at conflict emergence but loses the conflict's-emergence aspect after forgetting; provenance for the resolution outcome may persist but the provenance linking that outcome back to the originating conflict — and to the alternatives that the resolution chose against — is lost.

Cascade-implications. A2.36–A2.39 (the four accountability questions) are partially compromised for historical decisions: "what was decided" is preserved (the resolution outcome) but "with what rationale" loses its conflict-context component after forgetting. A1.10 (determinism contract) is compromised at the historical-retracing layer because resolution outcomes that depended on the forgotten conflict cannot be re-derived from current substrate. A1.01 (human-governed) is compromised because humans exercising the inspect right (A2.01) cannot inspect historical conflicts after forgetting; the inspect right is preserved as architectural property but is hollow for the class of facts removed. A2.14 (cell-level resolution under orchestration rules) is partially legitimate: the resolution itself is consistent with A2.14, and the failure is the post-resolution removal of substrate state that the resolution operated on.

4. The failure mode

The substrate state in a deployment exhibiting detect-resolve-forget contains resolution outcomes but not the conflict history that informed them. The downstream consequences are operationally specific.

Resolution outcome detached from conflict context. Substrate state after resolution carries the outcome but not the conflict that produced it; subsequent readers see the outcome without the disagreement.

Historical inspection blind to resolved conflicts. A human exercising the inspect right can identify currently active conflicts but not what conflicts have been resolved during the deployment's lifecycle.

Accountability questions partially unanswerable for historical decisions. The four accountability questions per A2.36–A2.39 may be fully answerable for current state but not for historical decisions involving resolved conflicts; "with what rationale" loses its conflict-context component after the conflict is forgotten.

Audit trails fragmented at forget moments. Audit processes attempting to reconstruct deployment history encounter substrate state changes recorded except at forget moments, where edges and provenance are removed. The trail has gaps where the conflict-and-resolution segments used to be.

Substrate as outcome-store rather than coordination-history-store. A1.08 commits substrate as the source of truth for coordination, including (per A2.45) conflict information. Detect-resolve-forget reduces substrate to an outcome-store; coordination history is not preserved.

Recovery from misjudged resolutions is operationally constrained. When a resolution is later judged misjudged, recovery requires reconstructing the conflict context. The deployment can change resolutions going forward but cannot meaningfully re-examine past ones.

Resolution patterns become operationally invisible. Recurring disagreements between specific sources, recurring rule-application outcomes, and drift in which authorities prevail in which conflict types are not visible in substrate after forgetting.

5. The architectural correction

The correction operates through three commitments held together: A1.03's first-class status as persistent, A2.13's substrate-level preservation as throughout-deployment, and A2.17's contradiction-edge persistence even after resolution.

Persistent conflict preservation. Conflicts are substrate-preserved throughout the deployment's existence. The contradiction edge per A2.17 produced at conflict emergence persists in substrate after resolution; it is not removed.

Status-marking in place of removal. Resolution marks the contradiction edge with appropriate status — "resolved by rule R via cell C," "resolved by human H," "resolved at time T" — and additional substrate fields recording the resolution outcome and its provenance. The contradiction edge persists with status indicators rather than being deleted. The architectural distinction is precise: status-marking is an enrichment of substrate state; removal is a deletion of substrate state.

Provenance for both emergence and resolution. Provenance per A2.16 records the conflict's emergence (the writes that conflicted, the sources, the time of detection); provenance per A2.40 records the resolution outcome with attribution to the cell, rule, and human authority that produced it. Both records persist; the conflict's complete history — its emergence, its resolution path, and the bridge between — is substrate-recorded.

Archival distinguished from forgetting. Archival processes that move resolved conflicts from active substrate to archival substrate are legitimate when (i) the archival substrate preserves the inspect right per A2.01, (ii) the archival relocation is itself substrate-recorded so the trail traces from active to archive, and (iii) the archival substrate's existence does not weaken any of the determinism guarantees per A1.10. The anti-pattern is specifically removal that erases conflict information from any substrate; archival that preserves access is consistent with the architecture.

Resolution context preserved as a coherent unit. The substrate state after resolution preserves the resolution context — conflict, resolving rule, resolution outcome, human authority — as a coherent architectural unit, addressable per A1.07 so downstream readers can retrieve full context from the resolution outcome backward through the conflict that produced it.

6. What detect-resolve-forget is NOT

Several adjacent patterns are commonly conflated with detect-resolve-forget. They are not the anti-pattern.

Not cell-level resolution under rules per A2.14 that preserves the resolved-conflict edge. Cell-level resolution where the resolved edge persists with status markers, resolution provenance, and outcome recording is legitimate. The anti-pattern arises specifically when the post-resolution edge is removed.

Not substrate state cleanup that does not remove conflict information. Cleanup of genuinely transient operational state — temporary buffers, expired authentication tokens, completed transient processes — is legitimate when it does not remove conflict information. The architectural commitment is to conflict preservation, not to maximal substrate retention.

Not archival processes that move conflicts to archival substrate while preserving inspectability. Archival that relocates resolved conflicts from active substrate to archival substrate is legitimate when the archival substrate preserves the inspect right and the archival relocation is itself substrate-recorded. Deletion that eliminates access is the failure; archival that preserves access is a legitimate substrate-management pattern.

Not status-marking resolved conflicts that retain edge presence. Substrate features that mark contradiction edges with status indicators ("resolved," "active," "pending," "deferred") are legitimate when the edges remain substrate-resident. The anti-pattern is status-marking-followed-by-removal.

7. Operational test

A deployment exhibits detect-resolve-forget if any of the following are observable at any point during the deployment's existence.

(a) Contradiction edges per A2.17 produced at conflict emergence are removed from substrate after resolution.

(b) Conflict-cleanup processes (UI-driven, workflow-driven, or schedule-driven) actively remove resolved conflicts from substrate.

(c) Garbage-collection logic removes conflicts older than time thresholds without preserving the conflict information in archival substrate.

(d) Substrate state after resolution contains only the resolution outcome without the conflict edge or provenance for emergence.

Three sharpening properties make the test operationally sufficient for deployment review.

Property (e.1) — Post-resolution-edge-presence test. Verify that contradiction edges persist after resolution. Simulate conflict emergence and resolution in a controlled deployment instance, then inspect substrate; absence of edges post-resolution is direct evidence of the anti-pattern.

Property (e.2) — Conflict-history-inspectability test. Verify that humans exercising the inspect right per A2.01 can identify what conflicts have been resolved during the deployment's lifecycle. Attempt historical conflict inspection over a non-trivial time horizon; inability to enumerate historical conflicts indicates the anti-pattern.

Property (e.3) — Resolution-context-completeness test. Verify that substrate state after resolution preserves the resolution context — conflict, resolving rule, resolution outcome, human authority — as a coherent unit. Examine post-resolution substrate; outcome-only states without the originating conflict context indicate the anti-pattern.

A deployment that satisfies any of (a)–(d) and any of (e.1)–(e.3) exhibits the anti-pattern.

One-sentence test. If a deployment's contradiction edges per A2.17 are removed from substrate after resolution — through conflict-cleanup processes, garbage collection, or resolution-as-replacement of the edge — and humans exercising the inspect right per A2.01 cannot identify what conflicts existed historically, the deployment exhibits *detect-resolve-forget*; A1.03 holds transiently but fails retroactively at A2.13's substrate-level preservation, with A2.17's contradiction edge directly violated by removal and A1.07's path retraceability broken at forget moments.

8. Conclusion

The CKS pattern's commitment to conflicts as first-class substrate objects (A1.03) is an architectural commitment, not an event-time commitment. *Detect-resolve-forget* names the failure mode in which the commitment is honored at conflict emergence and through resolution, then undone after resolution by removing the contradiction edge from substrate. The result is a substrate that contains resolution outcomes without the conflicts that produced them, a retraceable trail that breaks at forget moments, and an inspect right that is operationally hollow for historical conflicts. The architectural correction is precise: conflicts are preserved throughout the deployment's existence; resolution adds substrate state (status markers, resolution outcomes, resolution provenance) without subtracting the contradiction edge that the resolution operated on; archival processes that preserve inspectability are legitimate; deletion processes that erase conflict information are not. Status-marking is the architectural device that lets substrate indicate the inactive status of a resolved conflict without removing its existence.

A3.09 sits in a trio of conflict-handling anti-patterns at A1.03. A3.08 (silent conflict resolution) addresses the case where conflicts are never preserved at all — silent merge logic resolves conflicting writes without producing a contradiction edge. A3.09 (this note) addresses the case where conflicts are preserved transiently and then forgotten. A3.10 (contradiction collapse by automation) addresses the systematic-automation variant of A3.09. The three notes together cover the foundational A1.03 anti-patterns at three distinct temporal points: never-preserved (A3.08), preserved-then-forgotten (A3.09), and systematically-forgotten-at-scale (A3.10). Subsequent work that adopts the CKS pattern, deploys conflict-resolution surfaces over CKS substrates, or composes CKS with adjacent conflict-management or data-cleanup systems should treat *detect-resolve-forget* as named here.

Source paper

Li, W. (2026). *Coordination Outside the Model: A Human-Governed Substrate Pattern for AI Systems.* April 2026. ORCID: 0009-0004-8065-3235.

How to cite this note

Li, W. (2026). *Anti-Pattern: Detect-Resolve-Forget — Standalone Formalization of the Failure Mode Where Conflicts Are Detected and Resolved But Then Removed From Substrate, Violating Substrate-Level Preservation Retroactively in CKS.* May 6, 2026. ORCID: 0009-0004-8065-3235.